Heart Stroke Prediction Project

Date: May 2026

Table of Contents

1.1 Introduction 3
2.1 Data Preprocessing 4
3.1 Exploratory Data Analysis 6
3.2 Explanation of Findings 8
4.1 Data Visualization 8
5.1 Machine Learning Model 10
6.1 Model Evaluation 11
7.1 Interpretation and Discussion 12
7.2 Interpretation of Results 12
7.3 Model Performance 12
7.4 Improvements..……..13
8.1 Conclusion..... 13

1.1 Introduction

Heart stroke is considered one of the most dangerous health conditions because it may lead to serious complications and affect the patient's life significantly. Early prediction of stroke cases can help improve medical decisions and reduce health risks.

This project focuses on predicting whether a patient is likely to have a heart stroke using machine learning techniques. The dataset used in this project is a real-world medical dataset that contains 4238 records and 16 columns. It includes several health and lifestyle features such as age, gender, glucose level, BMI, blood pressure, cholesterol level, smoking habits, diabetes, and heart rate.

The problem addressed in this project is the difficulty of identifying stroke cases accurately based on patient medical information. Machine learning can help analyze these factors and detect patterns that may indicate the risk of stroke.

Several data science techniques were applied during this project, including data preprocessing, exploratory data analysis (EDA), data visualization, machine learning modeling, and model evaluation. The preprocessing stage included handling missing values, checking duplicates, and converting categorical values into numerical values so they can be used in machine learning algorithms. After that, exploratory analysis and visualization techniques were used to better understand the dataset and identify relationships between variables.

Logistic Regression was selected as the machine learning model in this project. The model was trained and tested using the dataset, and its performance was evaluated using different evaluation metrics such as accuracy score, confusion matrix, and classification report.

The main objective of this project is to analyze patient health data, predict stroke cases, evaluate the model performance, and discuss possible improvements for better prediction accuracy.

2.1 Data Preprocessing

In this step, the dataset was cleaned and prepared before applying machine learning techniques.

The following preprocessing steps were performed:

- Step 1: Checking Missing Values

```
[200]: df.isnull().sum()
```

[200]:

	0
Gender	0
age	0
education	105
currentSmoker	0
cigsPerDay	29
BPMeds	53
prevalentStroke	0
prevalentHyp	0
diabetes	0
totChol	50
sysBP	0
diaBP	0
BMI	19
heartRate	1
glucose	388
Heart_ stroke	0

dtype: int64

Figure 1: Missing Values Check

The dataset was checked for missing values in all columns. Some columns such as education, cigsPerDay, BPMeds, totChol, BMI, heartrate, and glucose, contained missing values that needed to be handled before applying machine learning techniques.

- Step 2: Handling Missing Values

```
columns = ["cigsPerDay", "BPMeds", "totChol", "BMI", "heartRate", "glucose"]

for col in columns:
    df[col] = df[col].fillna(df[col].median())

df["education"] = df["education"].fillna(df["education"].mode()[0])
```

Figure 2: Filling Missing Values

Missing numerical values were filled using the median, while the missing values in the education column were filled using the mode because it is a categorical feature.

- Step 3: Converting Categorical Data

```
df["Gender"] = df["Gender"].str.strip().str.lower().map({"male": 1, "female": 0})
df["Heart_ stroke"] = df["Heart_ stroke"].str.lower().map({"yes": 1, "no": 0})
```

Figure 3: Converting Categorical Data

Categorical values such as Male/Female and yes/no were converted into numerical values so they could be used in the machine learning model.

- Step 4: Encoding Education Column

```
df = pd.get_dummies(df, columns=["education"], drop_first=True)
df["prevalentStroke"] = df["prevalentStroke"].str.lower().map({"yes": 1, "no": 0})
```

Figure 4: Encoding Education Column

One-Hot Encoding was applied to the education column to convert categorical values into numerical columns suitable for machine learning algorithms.

- Step 5: Checking Duplicates

```
(df.duplicated().sum())

np.int64(0)
```

Figure 5: Duplicate Values Check

The dataset was checked for duplicate rows, and no duplicate values were found in the data. Therefore, no rows needed to be removed.

3.1 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to better understand the dataset and identify important patterns, relationships, and trends between variables. Statistical methods were used to summarize the data and provide insights into the dataset characteristics. This process also helped identify potential outliers and understand the distribution of the data.

The step performed in this task:

- Step 1: Descriptive Statistics

```
df.describe()
```

	Gender	age	currentSmoker	cigsPerDay	BPMeds	prevalentStroke	prevalentHyp
count	4238.000000	4238.000000	4238.000000	4238.000000	4238.000000	4238.000000	4238.000000
mean	0.429212	49.584946	0.494101	8.941482	0.029259	0.005899	0.310524
std	0.495022	8.572160	0.500024	11.902399	0.168552	0.076587	0.462763
min	0.000000	32.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	42.000000	0.000000	0.000000	0.000000	0.000000	0.000000
50%	0.000000	49.000000	0.000000	0.000000	0.000000	0.000000	0.000000
75%	1.000000	56.000000	1.000000	20.000000	0.000000	0.000000	1.000000
max	1.000000	70.000000	1.000000	70.000000	1.000000	1.000000	1.000000

Figure 6: Descriptive Statistics Using df.describe()

Descriptive statistics were used to summarize the dataset by calculating values such as the mean, minimum, maximum, and standard deviation for numerical features.

Step 2: Identifying Patterns and Relationships

```
df.groupby("Heart_ stroke")["age"].mean()

Heart_ stroke
0    48.767668
1    54.145963
Name: age, dtype: float64
```

Figure 7: Average Age by Heart Stroke Status

This analysis was used to compare the average age between people with and without heart stroke. The results show that people with heart stroke have a higher average age compared to people without heart stroke, which suggests that stroke cases are more common among older individuals.

- Step 3: Detecting Outliers

```
plt.boxplot(df["totChol"])
plt.title("Cholesterol Boxplot")
plt.show()
```

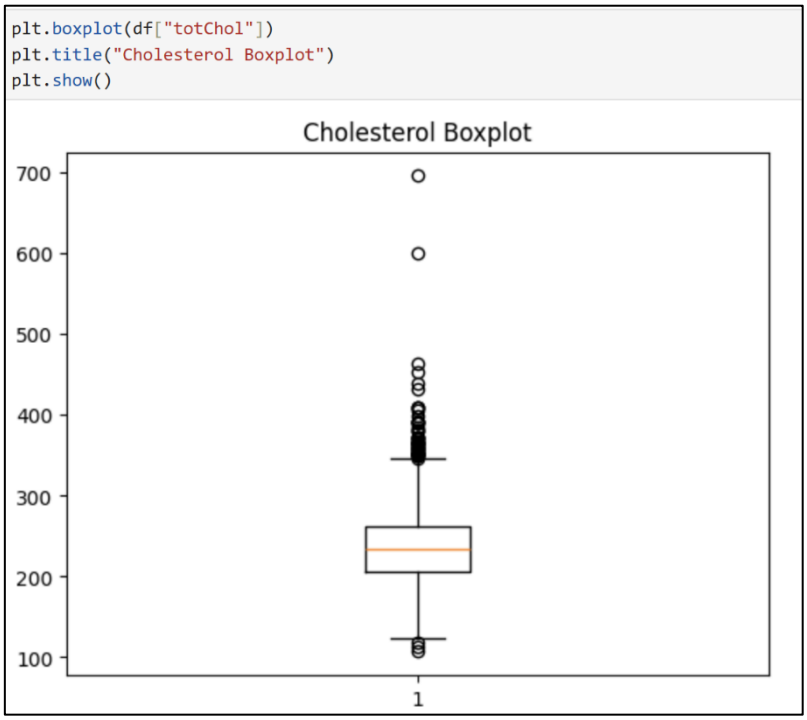


Figure 8: Cholesterol Box Plot

The box plot was used to detect outliers in cholesterol levels. The visualization shows the presence of unusually high cholesterol values in some patients.

3.2 Explanation of Findings

The analysis showed that several health-related factors such as age, cholesterol level, and blood pressure may affect the likelihood of heart stroke. In addition, some variables contained outliers and large value ranges that may influence the model performance.

4.1 Data Visualization

Different visualization techniques were used to better understand the dataset and identify important trends and patterns in the data. Visual representations help simplify the analysis and make relationships between variables easier to interpret.

The step performed in this task:

- Step 1: Histogram Visualization

```
plt.hist(df["age"])
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Count")
plt.show()
```

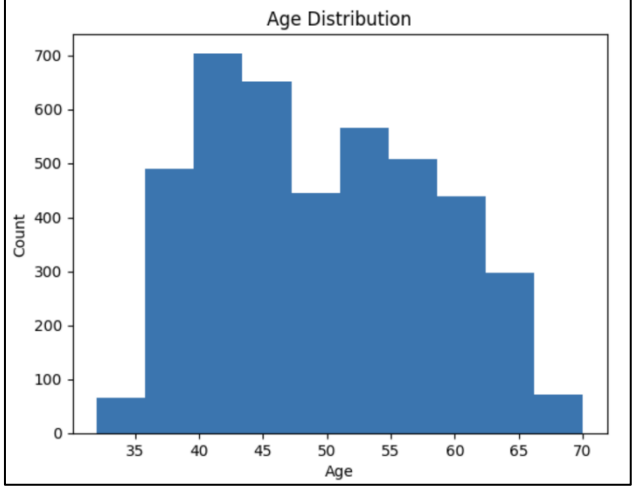


Figure 9: Age Distribution Histogram

The histogram shows the distribution of age values in the dataset. Most patients are middle-aged and older adults.

- Step 2: Bar Chart Visualization

```
df["currentSmoker"].value_counts().plot(kind="bar")
plt.title("Smoker vs Non-Smoker")
plt.xlabel("Smoking")
plt.ylabel("Count")
plt.show()
```

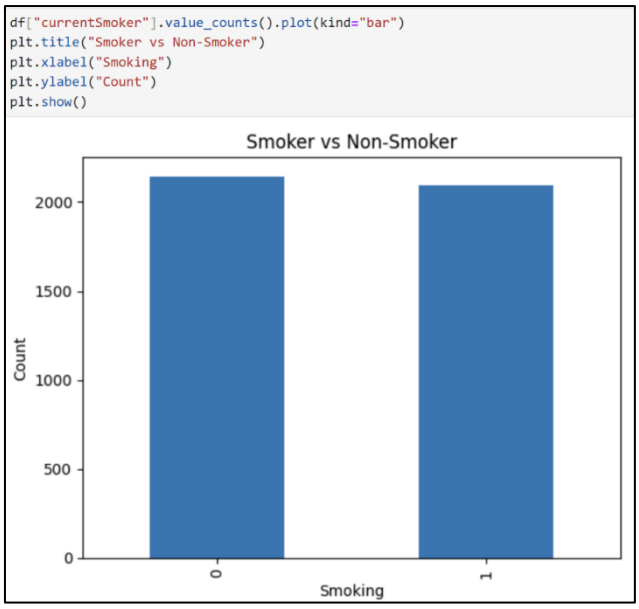


Figure 10: Smoker vs Non-Smoker Bar Chart

The bar chart compares the number of smokers and non-smokers in the dataset. The results show that both groups are relatively close in size.

- Step 3: Correlation Analysis

```
import seaborn as sns
import matplotlib.pyplot as plt

corr = df.corr()

plt.figure(figsize=(4,3))
sns.heatmap(corr, cmap="coolwarm")

plt.title("Correlation Heatmap")
plt.show()
```

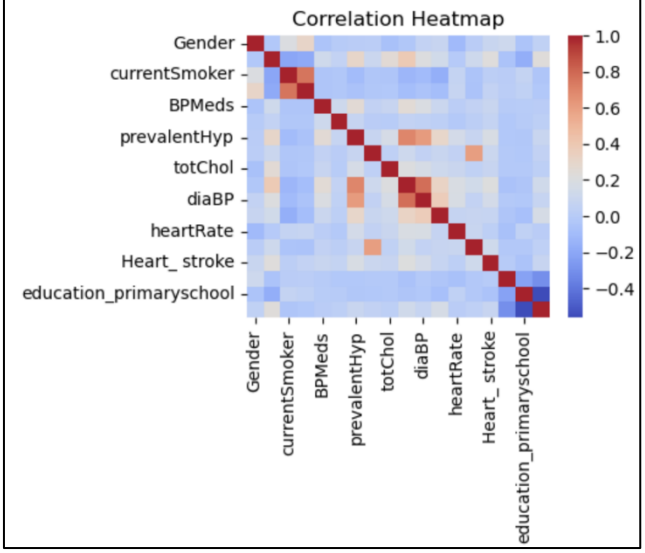


Figure 11: Correlation Heatmap of Dataset Features

This heatmap visualizes the correlation between variables in the dataset. Red colors indicate stronger positive relationships, while blue colors indicate weaker or negative relationships. The results show that some variables, such as smoking-related features and blood pressure measurements, have stronger correlations compared to other variables. Most features show weak to moderate relationships, which helps identify factors that may influence heart stroke prediction.

5.1 Machine Learning Model

A machine learning model was developed to predict whether a patient is likely to have a heart stroke based on health and medical information. Logistic Regression was selected as the classification model because it is suitable for binary classification problems.

The step performed in this task:

- Step 1: Defining Input and Output Variables

```
X = df.drop("Heart_ stroke", axis=1)
y = df["Heart_ stroke"]
```

Figure 12: Defining Features and Target Variable

The input features (X) were selected from all columns except the target column. The target variable (y) was the "Heart_ stroke" column, which represents whether the patient had a stroke or not.

- Step 2: Splitting the Dataset

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Figure 13: Splitting Training and Testing Data

The dataset was divided into training and testing sets using train_test_split(). The training data was used to train the model, while the testing data was used to evaluate its performance.

- Step 3: Training the Model

```
model = LogisticRegression(max_iter=2000)
model.fit(X_train, y_train)
```

Figure 14: Logistic Regression Model Training

The Logistic Regression model was trained using the training dataset to learn patterns and relationships between the features and the target variable.

- Step 4: Prediction Results

```
y_pred = model.predict(X_test)

print(y_pred[:10])
print(y_test[:10].values)
```

```
[0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 1 0 0 1]
```

Figure 15: Prediction Results

The prediction results show the model's predicted values compared with the actual values from the testing dataset. The model correctly predicted many cases, although some incorrect predictions were also observed.

6.1 Model Evaluation

Model evaluation was performed to measure the performance of the machine learning model and determine how accurately it predicts heart stroke cases. Different evaluation metrics were used to better understand the strengths and weaknesses of the model.

The step performed in this task:

- Step 1: Accuracy Score

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)
("Accuracy:", accuracy)
```

```
('Accuracy:', 0.8584905660377359)
```

Figure 16: Accuracy Score Result

The model achieved an accuracy of approximately 85.8%, which indicates good overall performance in predicting heart stroke cases.

- Step 2: Confusion Matrix

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)
(cm)
```

```
array([[718,   6],
       [114,  10]])
```

Figure 17: Confusion Matrix Result

The confusion matrix shows that the model predicts non-stroke cases more accurately than stroke cases. Some stroke cases were incorrectly predicted as non-stroke cases.

- Step 3: Classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.86      0.99      0.92       724
           1       0.62      0.08      0.14       124

    accuracy                           0.86       848
   macro avg       0.74      0.54      0.53       848
weighted avg       0.83      0.86      0.81       848
```

Figure 18: Classification Report Result

The classification report shows that the model performs very well for non-stroke cases, but the recall value for stroke cases is low. This means that the model struggles to correctly identify patients with stroke.

7.1 Interpretation and Discussion

The final step of the project focused on interpreting the model results and discussing the overall performance of the machine learning model. The strengths and weaknesses of the model were analyzed, and possible improvements were suggested.

7.2 Interpretation of Results

The model achieved an accuracy of approximately 85.8%, which indicates good overall performance in predicting heart stroke cases. The results show that the model performs well in predicting non-stroke cases.

7.3 Model Performance

Although the model achieved a high accuracy score, the confusion matrix and classification report revealed that the model struggles to correctly identify stroke cases. The recall value for stroke cases was very low, meaning that many actual stroke cases were predicted incorrectly.

This issue is likely caused by class imbalance in the dataset because non-stroke cases are much more common than stroke cases.

7.4 Improvements

Several improvements can be applied to enhance the model performance in future work, such as using balancing techniques like oversampling, trying different machine learning algorithms, increasing the amount of stroke case data, and adjusting model parameters for better prediction accuracy.

8.1 Conclusion

In conclusion, this project demonstrated the use of machine learning techniques to predict heart stroke cases based on patient health information. Several data science processes were applied, including data preprocessing, exploratory data analysis, data visualization, model building, and model evaluation. The Logistic Regression model achieved good overall accuracy in predicting non-stroke cases; however, the results also showed that the model had difficulty correctly identifying stroke cases due to the imbalance in the dataset. Overall, this project provided a better understanding of how machine learning can be used in healthcare prediction and how data analysis can support medical decision-making and future improvements in prediction systems